

# Rapport de projet — Microservices Docker/Kubernetes (objectif 20/20)

Binôme : *Nom 1* / *Nom 2*

Février 2026

## Table des matières

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Contexte et objectif</b>                                | <b>3</b> |
| <b>2</b> | <b>Vue d'ensemble du projet</b>                            | <b>3</b> |
| 2.1      | Architecture . . . . .                                     | 3        |
| 2.2      | Composants . . . . .                                       | 3        |
| 2.3      | Structure du dépôt . . . . .                               | 3        |
| <b>3</b> | <b>Fonctionnalités réalisées (démon)</b>                   | <b>4</b> |
| 3.1      | Catalogue . . . . .                                        | 4        |
| 3.2      | Commandes . . . . .                                        | 4        |
| 3.3      | UI . . . . .                                               | 4        |
| <b>4</b> | <b>Exécution locale (Docker Compose)</b>                   | <b>4</b> |
| 4.1      | Pré-requis . . . . .                                       | 4        |
| 4.2      | Lancement . . . . .                                        | 4        |
| 4.3      | Tests rapides (preuve fonctionnelle) . . . . .             | 4        |
| 4.4      | Test "gRPC observable" (produit inconnu) . . . . .         | 5        |
| 4.5      | Preuves à capturer (screenshots conseillés) . . . . .      | 5        |
| <b>5</b> | <b>Images Docker et publication Docker Hub</b>             | <b>5</b> |
| 5.1      | Build local (vérification) . . . . .                       | 5        |
| 5.2      | Push Docker Hub (reproductible) . . . . .                  | 5        |
| <b>6</b> | <b>Déploiement Kubernetes (Minikube)</b>                   | <b>5</b> |
| 6.1      | Pré-requis . . . . .                                       | 5        |
| 6.2      | Démarrage du cluster . . . . .                             | 5        |
| 6.3      | Déploiement des manifests . . . . .                        | 6        |
| 6.4      | Accès via Ingress . . . . .                                | 6        |
| 6.5      | Vérifications (commandes d'évaluation) . . . . .           | 6        |
| 6.6      | Alternative si l'Ingress n'est pas accessible . . . . .    | 6        |
| 6.7      | Preuves à capturer (screenshots conseillés) . . . . .      | 6        |
| <b>7</b> | <b>Base de données PostgreSQL (StatefulSet + PVC)</b>      | <b>7</b> |
| 7.1      | Justification . . . . .                                    | 7        |
| 7.2      | Vérification de la persistance et inspection SQL . . . . . | 7        |
| 7.3      | Preuves à capturer (screenshots conseillés) . . . . .      | 7        |

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>8</b>  | <b>Sécurité Kubernetes</b>                                     | <b>7</b>  |
| 8.1       | RBAC (ServiceAccount + Role + RoleBinding) . . . . .           | 7         |
| 8.2       | NetworkPolicy (segmentation réseau) . . . . .                  | 7         |
| 8.3       | Preuves à capturer (screenshots conseillés) . . . . .          | 8         |
| <b>9</b>  | <b>Bonus : gRPC</b>                                            | <b>8</b>  |
| 9.1       | Pourquoi gRPC ici ? . . . . .                                  | 8         |
| 9.2       | Commandes de démonstration . . . . .                           | 8         |
| <b>10</b> | <b>Bonus : Service Mesh Istio + mTLS STRICT</b>                | <b>8</b>  |
| 10.1      | Installation . . . . .                                         | 8         |
| 10.2      | Activation mTLS (STRICT) + routage via Istio Gateway . . . . . | 9         |
| 10.3      | Vérifications . . . . .                                        | 9         |
| <b>11</b> | <b>Pourquoi ce projet correspond à un niveau 20/20</b>         | <b>9</b>  |
| 11.1      | Intégration complète des technologies . . . . .                | 9         |
| 11.2      | Codage . . . . .                                               | 9         |
| 11.3      | Fonctionnalités . . . . .                                      | 9         |
| 11.4      | Présentation . . . . .                                         | 9         |
| <b>12</b> | <b>Checklist “jour de démo” (résumé commandes)</b>             | <b>10</b> |
| <b>13</b> | <b>Annexe : emplacements des captures d’écran</b>              | <b>10</b> |
| <b>14</b> | <b>Compilation du PDF</b>                                      | <b>10</b> |

# 1 Contexte et objectif

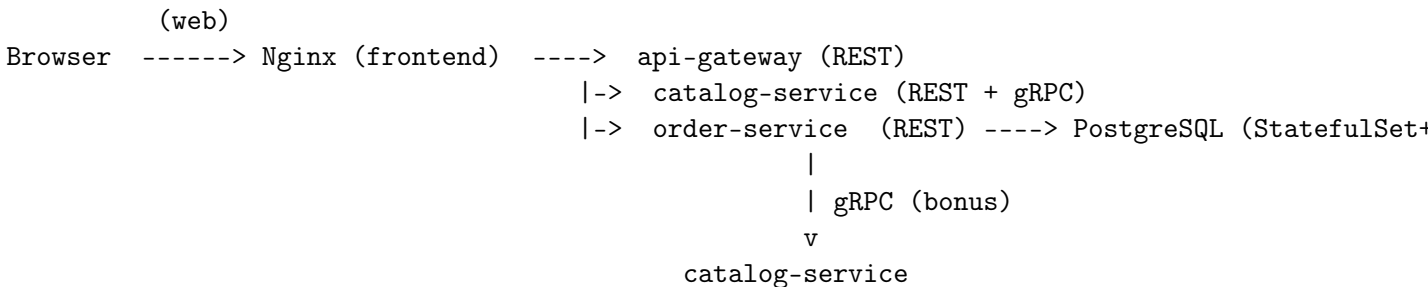
Le sujet impose la mise en place d'une architecture **microservices** avec des patterns et technologies modernes :

- microservices en **REST** (bonus : **gRPC**) ;
- conteneurisation **Docker** ;
- orchestration **Kubernetes** (Minikube) ;
- **Gateway** (Ingress Kubernetes et/ou Service Mesh) ;
- ajout d'une **base de données** ;
- **sécurisation** : RBAC, politiques réseau, et (bonus) chiffrement mTLS via Istio ;
- optionnel : déploiement cloud ;
- front-end (bonus d'ergonomie et de présentation).

**Objectif du dépôt livré** : fournir une démonstration reproductible “de bout en bout” (UI → Gateway → microservices → DB) couvrant un maximum de critères, et correspondant à un niveau **20/20** au regard de la grille annoncée (intégration techno, codage, fonctionnalités, présentation).

## 2 Vue d'ensemble du projet

### 2.1 Architecture



### 2.2 Composants

- **Frontend** : React (Vite) servi par **Nginx**. L'UI consomme `/api/*` (reverse proxy Nginx vers la gateway).
- **API Gateway** : Node.js/Express + `http-proxy-middleware`. Route `/api/products` → `catalog-service` et `/api/orders` → `order-service`.
- **catalog-service** : microservice Node/Express exposant :
  - REST : catalogue de produits (in-memory) ;
  - gRPC (bonus) : `CatalogService` (GetProduct, ListProducts).
- **order-service** : microservice Node/Express exposant un CRUD minimal sur des commandes :
  - persistance **PostgreSQL** (table `orders` + `order_items`) ;
  - appel **gRPC** vers `catalog-service` pour valider/enrichir les lignes de commande (bonus).
- **Kubernetes** : Deployments, Services, Ingress, Secret, StatefulSet + PVC, RBAC, NetworkPolicy.
- **Service mesh (bonus)** : Istio (Gateway/VirtualService + mTLS strict + DestinationRules).

### 2.3 Structure du dépôt

- `services/catalog-service` : microservice catalogue (+ proto gRPC).
- `services/order-service` : microservice commandes (+ DB + client gRPC).
- `gateway` : gateway API.
- `frontend` : application React + Nginx.
- `k8s` : manifests Kubernetes.
- `docker-compose.yml` : exécution locale (stack complète).
- `scripts` : push Docker Hub, aide patch images.

## 3 Fonctionnalités réalisées (démon)

### 3.1 Catalogue

- GET /products : liste des produits.
- GET /products/:id : détail d'un produit.
- GET /health : endpoint de santé (utilisé par les probes).

### 3.2 Commandes

- POST /orders : création d'une commande avec validation des produits via gRPC.
- GET /orders : liste des commandes + items.
- GET /orders/:id : détail d'une commande.
- GET /health : santé applicative + test DB.

### 3.3 UI

- Affichage du catalogue et saisie des quantités (panier).
- Création d'une commande.
- Affichage des commandes persistées en base (preuve de persistance).

## 4 Exécution locale (Docker Compose)

### 4.1 Pré-requis

- Docker + plugin `docker compose`
- (Optionnel) `jq` pour lire joliment les JSON.

### 4.2 Lancement

À la racine du projet :

```
docker compose up --build
```

Accès :

- UI : <http://localhost:8080/>
- API via Nginx (reverse proxy vers la gateway) :
  - <http://localhost:8080/api/products>
  - <http://localhost:8080/api/orders>
  - <http://localhost:8080/health>

### 4.3 Tests rapides (preuve fonctionnelle)

```
# Liste des produits
curl -s http://localhost:8080/api/products | jq

# Creation d'une commande
curl -s -X POST http://localhost:8080/api/orders \
  -H 'content-type: application/json' \
  -d '{"items":[{"productId":"p1","quantity":2},{"productId":"p2","quantity":1}]}' | jq

# Lecture des commandes (doit contenir l'ordre creee)
curl -s http://localhost:8080/api/orders | jq
```

## 4.4 Test "gRPC observable" (produit inconnu)

La validation des produits a lieu via gRPC. On le constate via le comportement :

```
# Doit retourner une erreur UNKNOWN_PRODUCT (400)
curl -i -s -X POST http://localhost:8080/api/orders \
  -H 'content-type: application/json' \
  -d '{"items":[{"productId":"UNKNOWN","quantity":1}]}'
```

## 4.5 Preuves à capturer (screenshots conseillés)

-  UI (catalogue + commandes).
-  `docker compose ps` avec les 5 conteneurs up.
-  sortie `curl` (OK + erreur produit inconnu).
-  logs `docker compose logs -f order-service` montrant création de commande (insert DB).

# 5 Images Docker et publication Docker Hub

Le projet fournit des Dockerfiles pour chaque composant (frontend, gateway, 2 services). Un script de push est inclus.

## 5.1 Build local (vérification)

```
docker build -t catalog-service:dev ./services/catalog-service
docker build -t order-service:dev ./services/order-service
docker build -t api-gateway:dev ./gateway
docker build -t frontend:dev ./frontend
```

## 5.2 Push Docker Hub (reproductible)

```
docker login
export DOCKERHUB_USER=<votre_user_dockerhub>

# Publie 4 images tagées 1.0.0
bash scripts/push.sh 1.0.0
```

*Note* : dans les manifests Kubernetes, les images sont référencées sous la forme `<dockerhub_user>/<image>:<tag>` (exemple : `illiesdhub/catalog-service:1.0.0`).

# 6 Déploiement Kubernetes (Minikube)

## 6.1 Pré-requis

- minikube
- kubectl
- addon Ingress Minikube

## 6.2 Démarrage du cluster

```
minikube start
minikube addons enable ingress
```

## 6.3 Déploiement des manifests

```
kubectl apply -f k8s/00-namespace.yaml

kubectl apply -n micro-20 -f k8s/10-postgres.yaml
kubectl apply -n micro-20 -f k8s/20-catalog.yaml
kubectl apply -n micro-20 -f k8s/30-order.yaml
kubectl apply -n micro-20 -f k8s/40-gateway.yaml
kubectl apply -n micro-20 -f k8s/45-frontend.yaml
kubectl apply -n micro-20 -f k8s/50-ingress.yaml

# Securite
kubectl apply -n micro-20 -f k8s/60-rbac.yaml
kubectl apply -n micro-20 -f k8s/70-networkpolicy.yaml
```

## 6.4 Accès via Ingress

L’Ingress est défini sur `micro.local`. Il faut mapper le host vers l’IP Minikube :

```
MINIKUBE_IP=$(minikube ip)
echo "$MINIKUBE_IP micro.local" | sudo tee -a /etc/hosts
```

Ensuite :

- UI : <http://micro.local/>
- API : <http://micro.local/api/products> et <http://micro.local/api/orders>

## 6.5 Vérifications (commandes d’évaluation)

```
kubectl get all -n micro-20
kubectl get ingress -n micro-20
kubectl describe ingress micro-ingress -n micro-20

# Logs utiles
kubectl logs deploy/api-gateway -n micro-20
kubectl logs deploy/order-service -n micro-20
kubectl logs deploy/catalog-service -n micro-20

# Probes/health
kubectl port-forward svc/api-gateway 8081:8081 -n micro-20
curl -s http://localhost:8081/health
```

## 6.6 Alternative si l’Ingress n’est pas accessible

```
# UI
kubectl port-forward svc/frontend 8080:80 -n micro-20
# API
kubectl port-forward svc/api-gateway 8081:8081 -n micro-20
```

## 6.7 Preuves à capturer (screenshots conseillés)

- 📎 `kubectl get pods -n micro-20` (tous Running/Ready).
- 📎 `kubectl get ingress -n micro-20` + accès navigateur sur `micro.local`.
- 📎 logs de `order-service` lors d’une création d’order.

## 7 Base de données PostgreSQL (StatefulSet + PVC)

### 7.1 Justification

La base est utilisée uniquement par `order-service` (pattern **database per service**). Elle est déployée dans Kubernetes via un **StatefulSet** avec **volumeClaimTemplates** (PVC automatique) et un **Secret** pour les variables d'environnement.

### 7.2 Vérification de la persistance et inspection SQL

```
# Voir le PVC cree
kubectl get pvc -n micro-20

# Se connecter a Postgres
kubectl exec -it statefulset/postgres -n micro-20 -- \
  psql -U app -d orders

# Dans psql :
# \dt
# SELECT * FROM orders ORDER BY created_at DESC LIMIT 5;
# SELECT * FROM order_items LIMIT 10;
```

### 7.3 Preuves à capturer (screenshots conseillés)

-  `kubectl get pvc -n micro-20` (preuve volume).
-  requête SQL montrant des lignes `orders` et `order_items`.

## 8 Sécurité Kubernetes

### 8.1 RBAC (ServiceAccount + Role + RoleBinding)

Les pods applicatifs tournent sous un **ServiceAccount** dédié `app-sa`. Les permissions sont volontairement minimales : lecture de ressources nécessaires à l'observabilité/service discovery (Services, Endpoints, Pods, ConfigMaps) dans le namespace.

Commandes de vérification :

```
# Verifier le SA utilise par les pods
kubectl get pod -n micro-20 -o jsonpath='{range .items[*]}{.metadata.name}{ " -> "}{.spec.serviceAccountName}{ "\n"}{end}'

# Test d'autorisation (impersonation)
kubectl auth can-i list pods -n micro-20 --as=system:serviceaccount:micro-20:app-sa
kubectl auth can-i delete pods -n micro-20 --as=system:serviceaccount:micro-20:app-sa
```

Résultat attendu :

-  `list pods : yes`
-  `delete pods : no`

### 8.2 NetworkPolicy (segmentation réseau)

Le projet applique :

- un **deny-all** ingress par défaut ;
- des exceptions explicites : Ingress → frontend, frontend → gateway, gateway → services, order-service → postgres + gRPC vers catalog-service ;
- autorisation d'egress DNS.

**Attention** : NetworkPolicy nécessite un CNI compatible (Calico/Cilium). Exemple avec Calico :

```
# Option 1 : (re)créer le cluster minikube avec Calico
minikube delete
minikube start --network-plugin=cni --cni=calico
minikube addons enable ingress
```

### 8.3 Preuves à capturer (screenshots conseillés)

-  `kubectl get role,rolebinding,sa -n micro-20.`
-  `kubectl get networkpolicy -n micro-20.`
-  (optionnel) test de flux autorisé / refusé via pods debug.

## 9 Bonus : gRPC

### 9.1 Pourquoi gRPC ici ?

- Communication inter-services typée, performante, et contractuelle via `proto`.
- Le `order-service` dépend du `catalog-service` pour valider l'existence et le prix du produit au moment de la commande.

### 9.2 Commandes de démonstration

```
# En local (Docker Compose) :
curl -s -X POST http://localhost:8080/api/orders \
  -H 'content-type: application/json' \
  -d '{"items":[{"productId":"p1","quantity":1}]} ' | jq

# Produit inexistant -> preuve de validation inter-service
curl -i -s -X POST http://localhost:8080/api/orders \
  -H 'content-type: application/json' \
  -d '{"items":[{"productId":"bad","quantity":1}]} '
```

Optionnel (preuve directe) : utiliser `grpcurl` avec un port-forward sur `catalog-service` :

```
kubectl port-forward svc/catalog-service 50051:50051 -n micro-20
grpcurl -plaintext localhost:50051 list
grpcurl -plaintext localhost:50051 catalog.CatalogService/ListProducts
```

## 10 Bonus : Service Mesh Istio + mTLS STRICT

### 10.1 Installation

```
# Pre-requis : istioctl installe
istioctl install -y --set profile=demo

# Injection sidecar dans le namespace
kubectl label namespace micro-20 istio-injection=enabled --overwrite

# Redémarrage des deployments pour injecter Envoy
kubectl rollout restart deploy -n micro-20 catalog-service order-service api-gateway
frontend
```



## 10.2 Activation mTLS (STRICT) + routage via Istio Gateway

```
kubectl apply -n micro-20 -f k8s/istio/00-peerauth-strict.yaml
kubectl apply -n micro-20 -f k8s/istio/10-destinationrules.yaml
kubectl apply -n micro-20 -f k8s/istio/20-gateway.yaml
kubectl apply -n micro-20 -f k8s/istio/30-virtualservice.yaml
```

## 10.3 Vérifications

```
# Verifier que les pods ont 2 containers (app + istio-proxy)
kubectl get pods -n micro-20

# Verifier la config cote mesh
istioctl proxy-status

# (Optionnel) verifier la strategie mTLS appliquee
kubectl get peerauthentication -n micro-20
```

# 11 Pourquoi ce projet correspond à un niveau 20/20

Au regard des critères (par ordre décroissant) :

### 11.1 Intégration complète des technologies

- ✓ Microservices : 2 services métiers + gateway + frontend.
- ✓ REST sur chaque service + endpoints health + probes K8s.
- ✓ Bonus **gRPC** inter-services (proto + client + serveur).
- ✓ Docker : Dockerfiles pour chaque composant + stack Docker Compose.
- ✓ Kubernetes : Deployments, Services, Ingress, StatefulSet, Secrets, PVC.
- ✓ Sécurité : RBAC + NetworkPolicy.
- ✓ Bonus **Istio/service mesh** : Gateway/VirtualService + mTLS STRICT.

### 11.2 Codage

- ✓ Node/Express structuré (fichiers dédiés : DB, validation, client gRPC).
- ✓ Validation d'entrée (Zod), gestion d'erreurs, transactions SQL.
- ✓ Configuration 12-factor via variables d'environnement.

### 11.3 Fonctionnalités

- ✓ Parcours complet : consulter catalogue → créer commande → persistance DB → consulter commandes.
- ✓ Cas d'erreur métier : refus d'une commande avec produit inconnu (preuve d'appel inter-service).

### 11.4 Présentation

- ✓ Front React avec UI claire (catalogue, panier, commandes persistées).
- ✓ Reverse proxy Nginx (pattern front → API gateway).

## 12 Checklist “jour de démo” (résumé commandes)

```
# Local
docker compose up --build

# K8s
minikube start
minikube addons enable ingress
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -n micro-20 -f k8s/10-postgres.yaml
kubectl apply -n micro-20 -f k8s/20-catalog.yaml
kubectl apply -n micro-20 -f k8s/30-order.yaml
kubectl apply -n micro-20 -f k8s/40-gateway.yaml
kubectl apply -n micro-20 -f k8s/45-frontend.yaml
kubectl apply -n micro-20 -f k8s/50-ingress.yaml
kubectl apply -n micro-20 -f k8s/60-rbac.yaml
kubectl apply -n micro-20 -f k8s/70-networkpolicy.yaml

MINIKUBE_IP=$(minikube ip)
echo "$MINIKUBE_IP micro.local" | sudo tee -a /etc/hosts
```

## 13 Annexe : emplacements des captures d’écran

Pour rendre le rapport **autoportant**, on peut ajouter les captures dans un dossier `docs/screens/` puis les inclure ainsi :

```
\begin{figure}[h]
  \centering
  \includegraphics[width=\linewidth]{docs/screens/ui.png}
  \caption{UI --- Catalogue et commandes}
\end{figure}
```

## 14 Compilation du PDF

```
pdflatex rapport_microservices_20.tex
# ou
latexmk -pdf rapport_microservices_20.tex
```